MODELO PADRÃO DE ENTREGA DE ATIVIDADES

Etapa	Entrega final	**Atividade**	13
Encontro	13	**Curso**	ADS / ULBRA
Aluno	Vinicius Casagrande	**Professor(a)**	Caroline Pacheco da Rosa
Projeto	Recomendação de treinos personalizados em academia		

Artefato verificável

Repositório (GitHub): https://github.com/Vinicasagrande87/fundamentos_ia

Notebook do experimento: atividade13_academia.ipynb — executado do início ao fim, com baseline, árvore de decisão e rede neural treinados no mesmo notebook, todas as saídas e figuras visíveis (curva de perda, comparação lado a lado e matrizes de confusão).

Figuras salvas: curva_perda_atividade13.png e comparacao_modelos_atividade13.png (raiz do repositório).

README.md atualizado: seção "13. Atividade 13 — Rede neural x modelo clássico"; backlog da AS atualizado em entregas/Backlog_AS_ViniciusCasagrande.md.

Contexto

O projeto recomenda uma categoria de treino (A = musculação, B = cardio + circuito, C = funcional, D = peso corporal adaptado) para alunos de academia. O modelo clássico é uma Árvore de Decisão (max_depth=5), treinada sobre 300 alunos simulados (15 features depois do one-hot, 8% de ruído de rótulo proposital). A métrica principal, definida na Atividade 10, é o recall de Treino D: um falso negativo dessa classe manda um aluno com restrição, pouco equipamento ou pouca experiência para um treino mais intenso do que o indicado. Nesta atividade treinei uma rede neural (MLPClassifier) para descobrir, com evidência, se ela faz sentido neste projeto.

1. Experimento executado

1.1 Comparação justa

- **Mesmos dados:** mesma base tratada, mesma codificação (one-hot) e a mesma divisão 80/20 estratificada com random_state=42 das Atividades 9 e 10: 240 alunos no treino, 60 no teste (18 de Treino D);
- **Mesmo notebook:** baseline (DummyClassifier most_frequent), árvore de decisão e rede neural treinados no mesmo notebook, sobre o mesmo X_train, e avaliados no mesmo X_test;
- **Mesmas métricas:** recall de Treino D (métrica principal), acurácia e F1-macro, calculados pela mesma função para os três modelos;
- **Confirmação:** como o teste tem só 60 alunos, repeti a comparação com validação cruzada k=5 usando os mesmos folds para todos os modelos.

1.2 Padronização

O StandardScaler fica dentro de um Pipeline junto com a rede. Assim ele é ajustado só no treino (e só no treino de cada fold da validação cruzada), sem vazamento para o teste. Também deixei registrada uma rede treinada sem padronização: ela ficou em 0,60 de acurácia e não convergiu, o que mostra na prática por que o scaler é obrigatório. Essa rede não entra na comparação.

1.3 Escolha e registro da arquitetura

Para não escolher a arquitetura olhando o teste, comparei 6 candidatas com validação cruzada de 5 folds feita só dentro do treino:

Camadas escondidas	Parâmetros	Acurácia treino	Acurácia validação	Recall D validação
(4,)	84	0,848	0,771	0,717
(8,)	164	0,938	0,742	0,690
(16,) ← escolhida	**324**	**0,962**	**0,792**	**0,784**
(32,)	644	0,982	0,792	0,823
(16, 8)	428	0,983	0,767	0,838
(64, 32)	3236	0,994	0,779	0,825

(16,) e (32,) empataram na validação (0,792). Fiquei com a (16,), que tem metade dos parâmetros, porque com 240 alunos de treino cada parâmetro a mais é mais chance de decorar ruído. As redes de duas camadas chegam a 98-99% no treino e pioram na validação, o que já é um sinal de overfitting.

Item	Valor
Entrada	15 features (2 numéricas + 13 one-hot), padronizadas
Camadas escondidas	1
Neurônios	16
Ativação	ReLU na camada escondida; softmax na saída (4 neurônios: Treino A, B, C, D)
Otimizador	Adam, taxa de aprendizado 0,001, alpha (L2) 0,0001
Épocas	1479 (limite max_iter=3000; parou sozinha ao convergir, sem aviso)
Parâmetros ajustados	324, para 240 alunos de treino

2. Comparação com o modelo clássico e o baseline

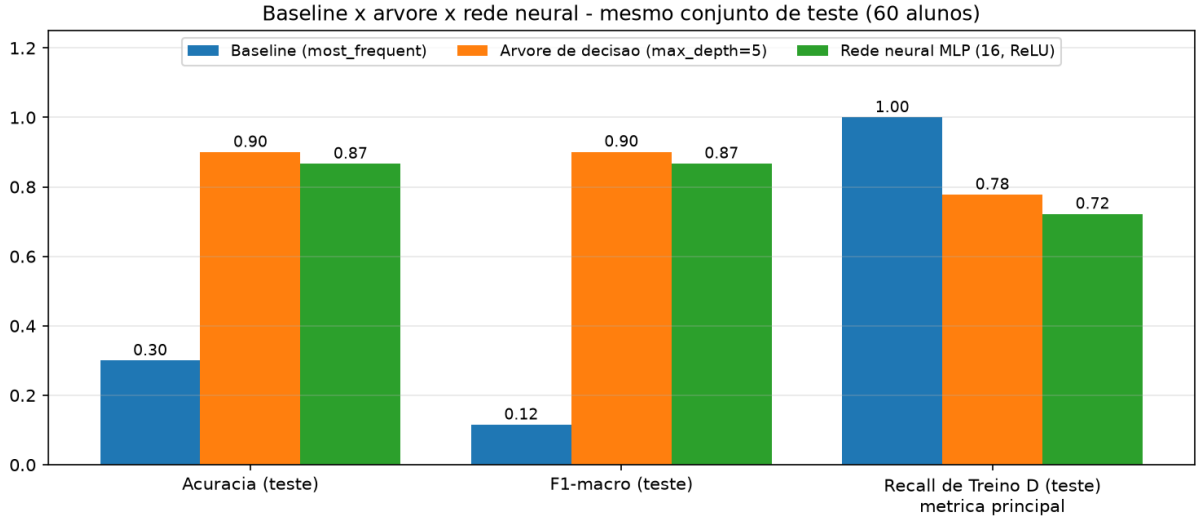


Figura 1 — Mesmo conjunto de teste (60 alunos) para os três modelos.

Modelo (split único, random_state=42)	Acur. treino	Acur. teste	F1-macro	Recall D (principal)
Baseline (most_frequent)	0,3083	0,3000	0,1154	1,0000
Árvore de decisão (max_depth=5)	**0,8875**	**0,9000**	**0,9010**	**0,7778**
Rede neural MLP (16, ReLU)	0,9458	0,8667	0,8673	0,7222

Modelo (validação cruzada, mesmos 5 folds)	Acur. treino	Acur. teste	Desvio	Recall D
Baseline (most_frequent)	0,3067	0,3067	0,0082	1,0000
Árvore de decisão (max_depth=5)	0,9017	0,8267	0,0291	0,7310
Árvore balanced (max_depth=5)	0,9008	0,8400	0,0291	0,7637
Rede neural MLP (16, ReLU)	0,9567	0,8300	0,0245	0,8175

O baseline sempre responde "Treino D" (a classe mais frequente). Por isso o recall de D dele é 1,0, mas a acurácia é 0,30: ele não aprendeu nada. Isso mostra que o recall de D precisa ser lido junto com a acurácia. Tanto a árvore quanto a rede superam o baseline com folga, então as duas aprenderam o problema.

3. Análise do resultado

No split único, a rede piorou em relação à árvore: 0,87 contra 0,90 de acurácia e 0,72 contra 0,78 de recall de Treino D, no mesmo teste. Na validação cruzada, com os mesmos 5 folds, a rede empata em acurácia com a árvore original (0,830 contra 0,827; a diferença é menor que o desvio entre folds, de ~0,03) e fica abaixo da árvore com class_weight='balanced' (0,840). O único ponto em que a rede se destaca é o recall de D na validação cruzada (0,82

contra 0,73 e 0,76). Esse ganho, porém, não aparece no split único e equivale a 1 ou 2 alunos por fold, porque cada fold tem só ~18 alunos de Treino D.

Também testei se a rede corrigia o ponto cego encontrado na Atividade 12 (alunos com objetivo emagrecimento e restrição física). Não corrige: nesse grupo a árvore tem 7 falsos negativos reais de Treino D e a rede tem 6. A rede reduz os falsos negativos de D espalhados pela base (17 contra 25), mas não aprende a interação "restrição + iniciante → D" que falta.

Hipóteses técnicas:

- **Base pequena para a quantidade de parâmetros:** a rede escolhida ajusta 324 parâmetros com 240 alunos de treino, mais de um parâmetro por exemplo. A árvore com profundidade 5 tem no máximo 32 folhas. A tabela de arquiteturas mostra o efeito: quanto maior a rede, maior o acerto no treino e pior a validação.
- **Relações simples, em forma de regras de corte:** a base foi gerada por uma regra do tipo "sem equipamento → D; com restrição e iniciante ou até 2 dias → D; ...". São cortes por categoria e por limiar de dias, exatamente o formato de uma árvore. A rede precisa aproximar esses degraus com combinações de ReLUs, e isso exige mais dados.
- **Teto imposto pelo ruído de rótulo:** 8% dos rótulos contradizem a regra, o que limita a acurácia a ~0,92. Os dois modelos já estão perto desse teto no split único, e a capacidade que sobra na rede vai para decorar o ruído (0,95-0,96 de acurácia no treino).
- **O problema do recall de D está nas features:** o enunciado alerta que trocar de modelo raramente resolve um problema que está nos dados, e é o caso aqui: o ponto cego da Atividade 12 continua na rede. O que falta é informação explícita sobre a interação restrição × nível.

4. Treino × teste da rede neural

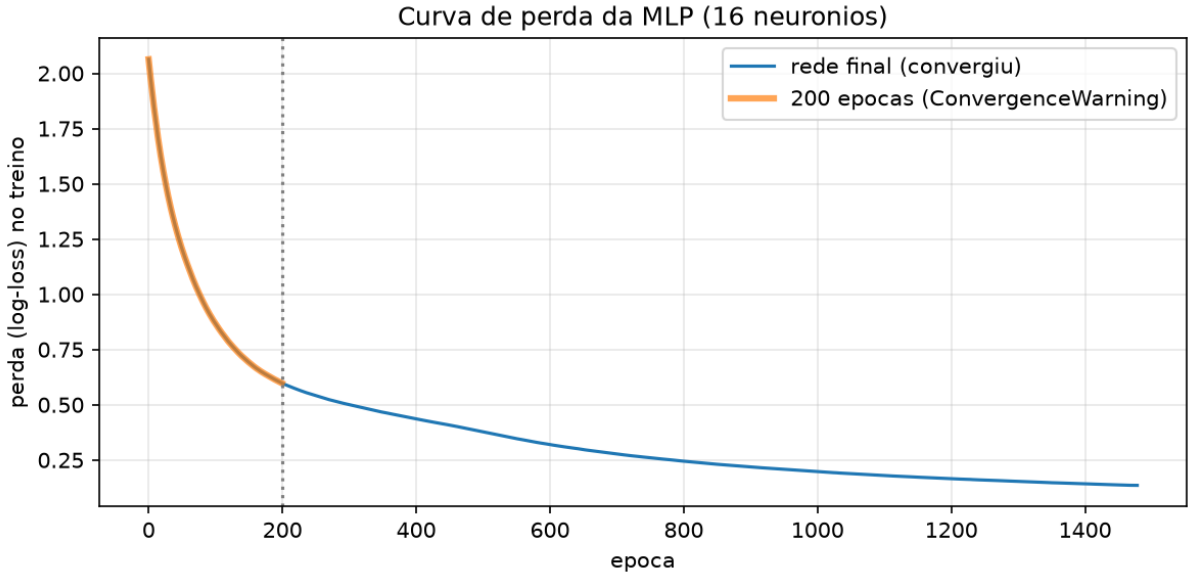


Figura 2 — Curva de perda no treino. Em laranja, a rede parada em 200 épocas (ConvergenceWarning); em azul, a rede final, que convergiu em 1479 épocas.

	Acurácia treino	Acurácia teste	Diferença
Rede neural (split único)	0,9458	0,8667	0,0792
Rede neural (validação cruzada)	0,9567	0,8300	0,1267
Árvore (split único)	0,8875	0,9000	-0,0125
Árvore (validação cruzada)	0,9017	0,8267	0,0750

A rede acerta bem mais no treino do que no teste: isso é overfitting. Na validação cruzada, a diferença dela (0,127) é quase o dobro da diferença da árvore (0,075), e no split único a árvore nem tem diferença. A rede está aprendendo particularidades dos 240 alunos de treino, incluindo os rótulos com ruído, que não se repetem em alunos novos. Na árvore, o max_depth=5 limita a capacidade e funciona como proteção contra isso.

5. Decisão sobre o projeto final

Decisão: Não vou usar rede neural no projeto final. O modelo da AS continua sendo a árvore de decisão (max_depth=5), e a versão com class_weight='balanced' passa a ser a candidata principal, porque teve a melhor acurácia na validação cruzada (0,84) sem perder interpretabilidade.

- **Base pequena e tabular (300 alunos, 15 features, 8% de ruído):** a rede não tem dados suficientes para justificar 324 parâmetros, e o resultado mostra isso: empate na validação cruzada, derrota no split único e overfitting maior.
- **A interpretabilidade é requisito do problema:** a recomendação envolve a segurança do aluno. As análises das Atividades 10 a 12 (regras com export_text, atalho por idade, ponto cego "restrição + iniciante") só foram possíveis porque a árvore mostra suas regras, e a regra de segurança proposta na Atividade 12 depende de entender por que o modelo errou. A rede esconde isso.
- **O ganho de recall de D da rede não compensa:** ele é pequeno (1 a 2 alunos por fold), não aparece no split único e não resolve o ponto cego. Para melhorar o recall de D, o caminho é a feature de interação restrição × iniciante e a regra de segurança, não trocar de modelo.

Conexão com a AS: Na AS, esta comparação entra no relatório final como a justificativa técnica para manter a árvore. O número de referência passa a ser a validação cruzada da árvore balanced (0,84), e a rede fica registrada como alternativa testada e descartada.

Organização, dificuldades e observações

- **ConvergenceWarning:** com o max_iter padrão (200), a rede não convergiu. A curva de perda mostra que a perda ainda estava caindo. Aumentei para max_iter=3000, e a rede parou sozinha em 1479 épocas pelo critério de parada do Adam, sem aviso. Nenhuma das 6 arquiteturas testadas nem os folds da validação cruzada geraram aviso com esse limite.
- **O resultado não convergido parecia melhor:** com 200 épocas a rede empatava com a árvore no teste (0,90) e tinha recall de D maior (0,89). Foi tentador usar esse número, mas ele vinha de um treino interrompido, não de uma escolha. Comparei apenas a rede convergida e deixei o outro resultado registrado como armadilha.
- **Padronização:** sem StandardScaler a rede ficou em 0,60. Coloquei o scaler num Pipeline para ele ser ajustado só no treino, inclusive dentro de cada fold.
- **Escolher a arquitetura sem contaminar o teste:** testar arquiteturas direto no teste e ficar com a melhor seria usar o teste para treinar. Por isso a escolha foi feita por validação cruzada dentro do treino.
- **Teste pequeno:** com 60 alunos no teste, cada aluno vale 1,7 ponto de acurácia. Por isso a decisão também se apoia na validação cruzada com os mesmos folds para todos os modelos.

Limitações: a base é sintética, então com alunos reais e mais dados a relação entre árvore e rede pode mudar. Só testei redes rasas com os hiperparâmetros padrão de otimização; uma busca maior (mais regularização, early_stopping) poderia reduzir o overfitting, mas dificilmente mudaria a conclusão com 300 alunos. O recall de D na validação cruzada tem alta variância, porque há poucos alunos de D por fold. Como efeito colateral, a validação cruzada da árvore balanced (0,84 ± 0,03), pendente no backlog desde a Atividade 11, foi feita aqui.

Declaração de uso de Inteligência Artificial

Uso de IA: Sim

Ferramentas utilizadas: Claude (Anthropic) / IA generativa

Como foi usada:

- apoio na estruturação e execução do código do notebook (Pipeline com StandardScaler, MLPClassifier, captura dos avisos de convergência, comparação de arquiteturas por validação cruzada no treino, validação cruzada com os mesmos folds para todos os modelos, gráficos);
- apoio na redação e formatação deste documento e da seção do README a partir dos números gerados no notebook;
- revisão de coerência entre os números do texto e as saídas do notebook.

Produção própria:

a definição de que a comparação seria feita pela métrica principal do projeto (recall de Treino D); a decisão de não usar o resultado não convergido de 200 épocas; a escolha da arquitetura (16,) em vez da (32,) pelo critério de menos parâmetros; a ligação com o ponto cego da Atividade 12; as hipóteses para o resultado; e a decisão de manter a árvore de decisão (com class_weight='balanced') no projeto final, justificada pelo tamanho da base, pelo ruído de rótulo e pela necessidade de interpretabilidade.